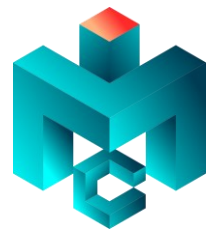




UNIVERSIDADE FEDERAL DE ITAJUBÁ
Instituto de Matemática e Computação



CMAC03 – Algoritmos em Grafos

4. Caminho Mínimo

Rafael Frinhani

frinhani@unifei.edu.br

2026

Apresentar os conceitos e algoritmos para obtenção de caminhos mínimos em grafos.

AGENDA

4. Caminho Mínimo

Contextualização, Algoritmo Guloso.

4.1. Algoritmo de Dijkstra

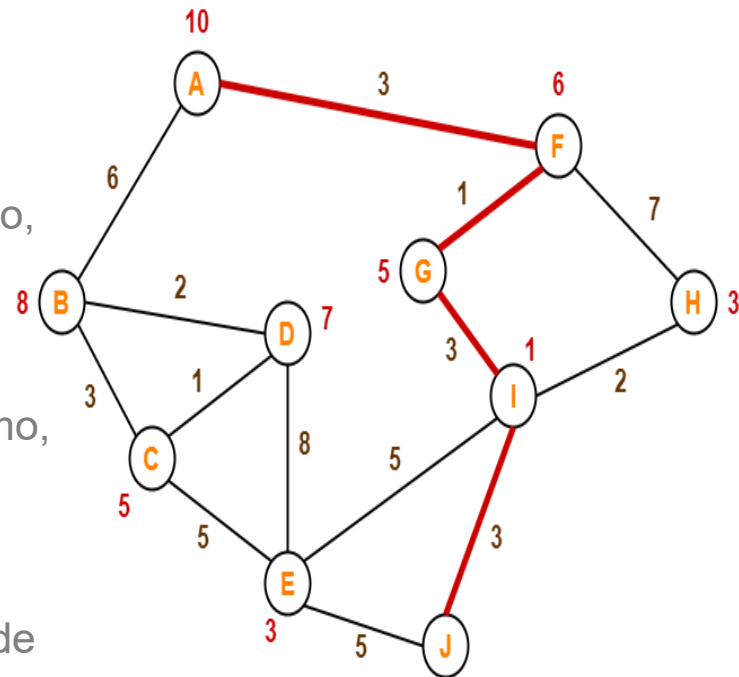
Definições, Princípio de Funcionamento, Algoritmo, Exemplo, Comentários, Limitação.

4.2. Algoritmo de Bellman-Ford

Definições, Princípio de Funcionamento, Algoritmo, Exemplo, Comentários.

4.3. Algoritmo de Floyd-Warshall

Definições, Princípio de Funcionamento, Ciclos de Custo Negativo, Algoritmo, Exemplo, Comentários, Composição de Caminhos.





4. Caminho Mínimo

Contextualização

O problema do caminho mínimo, ou caminho mais curto, consiste na **minimização do custo de travessia de um grafo** entre um vértice origem e um destino.

Em um **grafo não-ponderado**, o caminho mínimo entre dois vértices v (origem) e u (destino) é aquele que possui a **menor quantidade de arestas** entre eles (pode ser obtido com BFS).



4. Caminho Mínimo

Contextualização

O problema do caminho mínimo, ou caminho mais curto, consiste na minimização do custo de travessia de um grafo entre um vértice origem e um destino.

Em um grafo não-ponderado, o caminho mínimo entre dois vértices v (origem) e u (destino) é aquele que possui a menor quantidade de arestas entre eles (pode ser obtido com BFS).

No **grafo ponderado** leva-se em conta que o relacionamento entre os objetos não é idêntico (ex. distância física, tempo de traslado etc.). O custo da travessia a partir de um vértice v até alcançar u é dado pela **soma dos pesos de cada aresta (ou arco) percorrida**.

O caminho mínimo é aquele que possui o **menor custo entre todos os caminhos existentes entre v e u** . Poderá existir mais de um caminho mínimo entre v e u com custos mínimos iguais. O caminho mínimo pode não ser aquele com menor número de arestas.

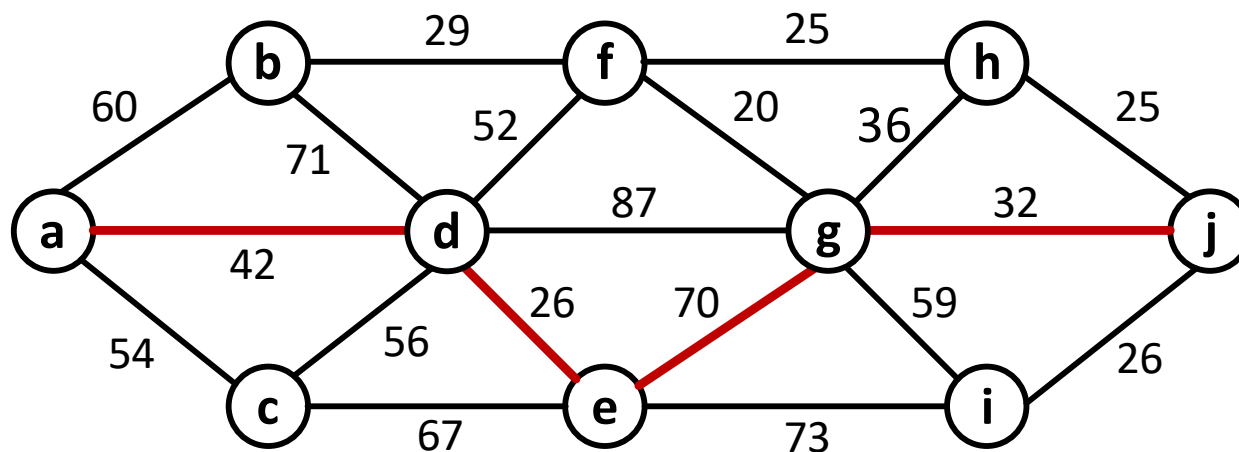


4. Caminho Mínimo

Algoritmo Guloso

Um algoritmo para o caminho mínimo baseado em uma estratégia gulosa **seleciona a aresta de menor custo a cada instante**.

A **estratégia gulosa restringe** a visão do custo do caminho para uma perspectiva **local**. Como o método não volta atrás na sua decisão, caminhos melhores poderão ser desconsiderados na análise.



Custo do Caminho

$$\begin{aligned} \{a, d\} &= 42 \\ \{d, e\} &= 26 \\ \{e, g\} &= 70 \\ \{g, j\} &= 32 \\ \hline &170 \end{aligned}$$

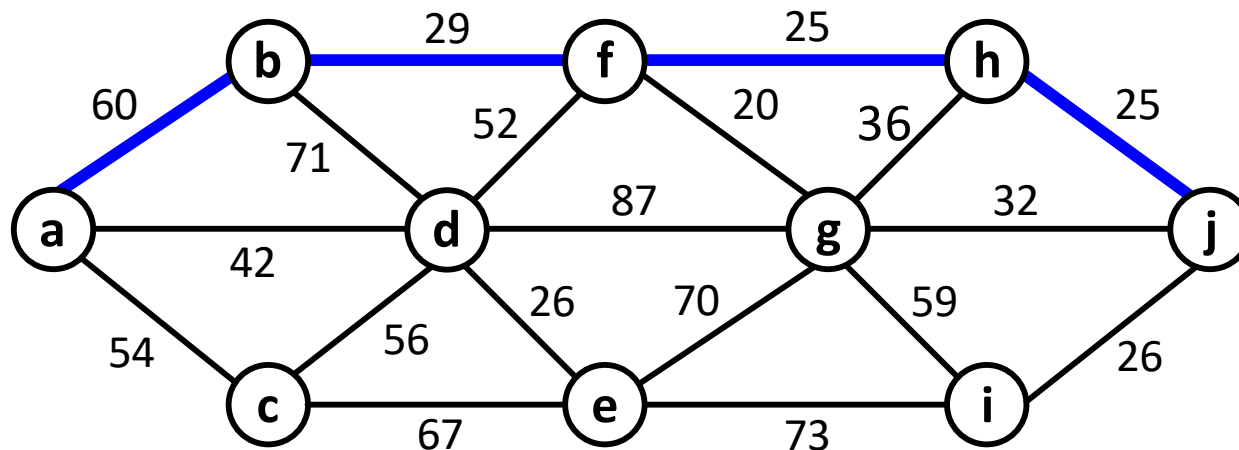


4. Caminho Mínimo

Algoritmo Guloso

Um algoritmo para o caminho mínimo baseado em uma estratégia gulosa seleciona a aresta de menor custo a cada instante.

A estratégia gulosa restringe a visão do custo do caminho para uma perspectiva local. Como o método não volta atrás na sua decisão, caminhos melhores poderão ser desconsiderados na análise.



Custo do Caminho

$$\begin{aligned}\{a, b\} &= 60 \\ \{b, f\} &= 29 \\ \{f, h\} &= 25 \\ \{h, j\} &= 25 \\ \hline &139\end{aligned}$$

Caminho Mínimo a – j
a, b, f, h, j



4. Caminho Mínimo

Algoritmo de Dijkstra

- O algoritmo proposto por Edsger Wybe Dijkstra em 1959.
- Aplicado a grafos simples ou dígrafos, e ponderados (apenas pesos positivos);
- Estruturalmente semelhante à BFS, adota o mesmo conceito de camadas:
 - Obtém a menor distância do vértice origem até seus vizinhos, depois dos vizinhos do vértice origem até seus próprios vizinhos e assim por diante.
 - Atualiza as distâncias sempre que descobre uma menor.
 - Atualiza o vértice precedente sempre que atualiza uma distância.



4. Caminho Mínimo

Algoritmo de Dijkstra

- O algoritmo proposto por Edsger Wybe Dijkstra em 1959.
- Aplicado a grafos simples ou dígrafos, e ponderados (apenas pesos positivos);
- Estruturalmente semelhante à BFS, adota o mesmo conceito de camadas:
 - Obtém a menor distância do vértice origem até seus vizinhos, depois dos vizinhos do vértice origem até seus próprios vizinhos e assim por diante.
 - Atualiza as distâncias sempre que descobre uma menor.
 - Atualiza o vértice precedente sempre que atualiza uma distância.

Princípio de Funcionamento

1. Mantém conjuntos de vértices abertos (A) e fechados (F). **Vértice Fechado** = **caminho mínimo do vértice origem até ele já foi calculado**. Caso contrário é considerado aberto.
2. Mantém o menor caminho conhecido até o momento para cada vértice descoberto.
3. Adiciona o vértice de menor caminho ao conjunto fechado.
4. Atualiza as distâncias considerando este vértice, descobrindo novos vértices.



4.1. Dijkstra

Algoritmo

dijkstra($G, v_{\text{Inicio}}, v_{\text{Fim}}$)

```
1 for  $i = 0$  até  $|V|$  do
2    $\text{custo}[i] = \infty$ ;
3    $\text{rota}[i] = v_{\text{Inicio}}$ ;
4 end
5  $\text{custo}[v_{\text{Inicio}}] = 0$ ;
6  $A = V$ ;
7  $F = \emptyset$ ;
8 while  $A \neq \emptyset$  do
9    $v = v \in A$  com menor  $\text{custo}[v]$  entre todos;
10   $F = F \cup \{v\}$ ;
11   $A = A - \{v\}$ ;
12   $N = N - F$ ;
13  for cada  $u \in N$  do
14    if  $\text{custo}[v] + w_{vu} < \text{custo}[u]$  then
15       $\text{custo}[u] = \text{custo}[v] + w_{vu}$ ;
16       $\text{rota}[u] = v$ ;
17    end
18  end
19 end
20  $\text{caminho} = \text{obtemCaminho}(\text{rota}, v_{\text{Inicio}}, v_{\text{Fim}})$ ;
21  $\text{caminho.reverse}$ ;
22 return  $\text{caminho}, \text{custo}[v_{\text{Fim}}]$ 
```

ESTRUTURAS

- F : conjunto de vértices Fechados.
- A : conjunto de vértices Abertos.
- N : conjunto de vértices vizinhos ao vértice atual.
- $\text{custo}[v]$: vetor que armazena o custo (distância) entre o vértice origem e os demais vértices de G .
- $\text{rota}[v]$: vetor que armazena o índice do vértice que precede o vértice v , no caminho cuja distância consta no vetor $\text{custo}[v]$.
- w_{vu} : valor do peso da aresta ou arco que conecta v e u .



4.1. Dijkstra

Algoritmo

dijkstra($G, vInicio, vFim$)

```
1 for  $i = 0$  até  $|V|$  do
2    $custo[i] = \infty$ ;
3    $rota[i] = vInicio$ ;
4 end
5  $custo[vInicio] = 0$ ;
6  $A = V$ ;
7  $F = \emptyset$ ;
8 while  $A \neq \emptyset$  do
9    $v = v \in A$  com menor  $custo[v]$  entre todos;
10   $F = F \cup \{v\}$ ;
11   $A = A - \{v\}$ ;
12   $N = N - F$ ;
13  for cada  $u \in N$  do
14    if  $custo[v] + w_{vu} < custo[u]$  then
15       $custo[u] = custo[v] + w_{vu}$ ;
16       $rota[u] = v$ ;
17    end
18  end
19 end
20  $caminho = obterCaminho(rota, vInicio, vFim)$ ;
21  $caminho.reverse$ ;
22 return  $caminho, custo[vFim]$ 
```

ESTRUTURAS

- F : conjunto de vértices Fechados.
- A : conjunto de vértices Abertos.
- N : conjunto de vértices vizinhos (adjacentes) ao vértice atual.
- $custo[v]$: vetor que armazena o custo (distância) entre o vértice origem e os demais vértices de G .
- $rota[v]$: vetor que armazena o índice do vértice que precede o vértice v , no caminho cuja distância consta no vetor $custo[v]$.
- w_{vu} : valor do peso da aresta ou arco que conecta v e u .

Os vetores de custo e rota são inicializados conforme $vInicio$ (linhas 1 a 4).

O custo de $vInicio$ é definido como 0 (linha 5), são inicializados os conjuntos A e F (linhas 6 e 7).



4.1. Dijkstra

Algoritmo

dijkstra($G, vInicio, vFim$)

```
1 for  $i = 0$  até  $|V|$  do
2    $custo[i] = \infty$ ;
3    $rota[i] = vInicio$ ;
4 end
5  $custo[vInicio] = 0$ ;
6  $A = V$ ;
7  $F = \emptyset$ ;
8 while  $A \neq \emptyset$  do
9    $v = v \in A$  com menor  $custo[v]$  entre todos;
10   $F = F \cup \{v\}$ ;
11   $A = A - \{v\}$ ;
12   $N = N - F$ ;
13  for cada  $u \in N$  do
14    if  $custo[v] + w_{vu} < custo[u]$  then
15       $custo[u] = custo[v] + w_{vu}$ ;
16       $rota[u] = v$ ;
17    end
18  end
19 end
20  $caminho = obterCaminho(rota, vInicio, vFim)$ ;
21  $caminho.reverse$ ;
22 return  $caminho, custo[vFim]$ 
```

ESTRUTURAS

- F : conjunto de vértices Fechados.
- A : conjunto de vértices Abertos.
- N : conjunto de vértices vizinhos (adjacentes) ao vértice atual.
- $custo[v]$: vetor que armazena o custo (distância) entre o vértice origem e os demais vértices de G .
- $rota[v]$: vetor que armazena o índice do vértice que precede o vértice v , no caminho cuja distância consta no vetor $custo[v]$.
- w_{vu} : valor do peso da aresta ou arco que conecta v e u .

Os vetores de custo e rota são inicializados conforme $vInicio$ (linhas 1 a 4).

O custo de $vInicio$ é definido como 0 (linha 5), são inicializados os conjuntos A e F (linhas 6 e 7).

Enquanto existirem vértices abertos (linhas 8 a 19):

Obter o vértice v a partir de A que tenha o menor custo entre todos os demais (linha 9).

Incluir v no conjunto F e remover do conjunto A (linhas 10 e 11).

Obter os adjacentes a v que ainda não foram fechados (linha 12)



4.1. Dijkstra

Algoritmo

dijkstra($G, vInicio, vFim$)

```
1 for  $i = 0$  até  $|V|$  do
2    $custo[i] = \infty$ ;
3    $rota[i] = vInicio$ ;
4 end
5  $custo[vInicio] = 0$ ;
6  $A = V$ ;
7  $F = \emptyset$ ;
8 while  $A \neq \emptyset$  do
9    $v = v \in A$  com menor  $custo[v]$  entre todos;
10   $F = F \cup \{v\}$ ;
11   $A = A - \{v\}$ ;
12   $N = N - F$ ;
13  for cada  $u \in N$  do
14    if  $custo[v] + w_{vu} < custo[u]$  then
15       $custo[u] = custo[v] + w_{vu}$ ;
16       $rota[u] = v$ ;
17    end
18  end
19 end
20  $caminho = obterCaminho(rota, vInicio, vFim)$ ;
21  $caminho.reverse$ ;
22 return  $caminho, custo[vFim]$ 
```

ESTRUTURAS

- F : conjunto de vértices Fechados.
- A : conjunto de vértices Abertos.
- N : conjunto de vértices vizinhos (adjacentes) ao vértice atual.
- $custo[v]$: vetor que armazena o custo (distância) entre o vértice origem e os demais vértices de G .
- $rota[v]$: vetor que armazena o índice do vértice que precede o vértice v , no caminho cuja distância consta no vetor $custo[v]$.
- w_{vu} : valor do peso da aresta ou arco que conecta v e u .

Para cada vértice u adjacente a v que ainda esteja em aberto (linha 13).

Se o custo de v mais o peso da aresta $\{v, u\}$ for menor que o custo de u (linha 14):

Atribuir como custo de u (linha 15) o custo do caminho até v mais o peso da aresta $\{v, u\}$.

Atribuir v como predecessor (antes) de u no vetor $rota$ (linha 16).



4.1. Dijkstra

Algoritmo

dijkstra($G, vInicio, vFim$)

```
1 for  $i = 0$  até  $|V|$  do
2    $custo[i] = \infty$ ;
3    $rota[i] = vInicio$ ;
4 end
5  $custo[vInicio] = 0$ ;
6  $A = V$ ;
7  $F = \emptyset$ ;
8 while  $A \neq \emptyset$  do
9    $v = v \in A$  com menor  $custo[v]$  entre todos;
10   $F = F \cup \{v\}$ ;
11   $A = A - \{v\}$ ;
12   $N = N - F$ ;
13  for cada  $u \in N$  do
14    if  $custo[v] + w_{vu} < custo[u]$  then
15       $custo[u] = custo[v] + w_{vu}$ ;
16       $rota[u] = v$ ;
17    end
18  end
19 end
20  $caminho = obterCaminho(rota, vInicio, vFim)$ ;
21  $caminho.reverse$ ;
22 return  $caminho, custo[vFim]$ 
```

ESTRUTURAS

- F : conjunto de vértices Fechados.
- A : conjunto de vértices Abertos.
- N : conjunto de vértices vizinhos (adjacentes) ao vértice atual.
- $custo[v]$: vetor que armazena o custo (distância) entre o vértice origem e os demais vértices de G .
- $rota[v]$: vetor que armazena o índice do vértice que precede o vértice v , no caminho cuja distância consta no vetor $custo[v]$.
- w_{vu} : valor do peso da aresta ou arco que conecta v e u .

Obtém o caminho mínimo percorrendo o vetor de rota a partir de $vFim$, passando pelos precedentes até alcançar $vInicio$ (linha 20).

Inverte o caminho mínimo (linha 21).

Retorna a sequência de vértices e o custo do caminho (linha 22).



4.1. Dijkstra

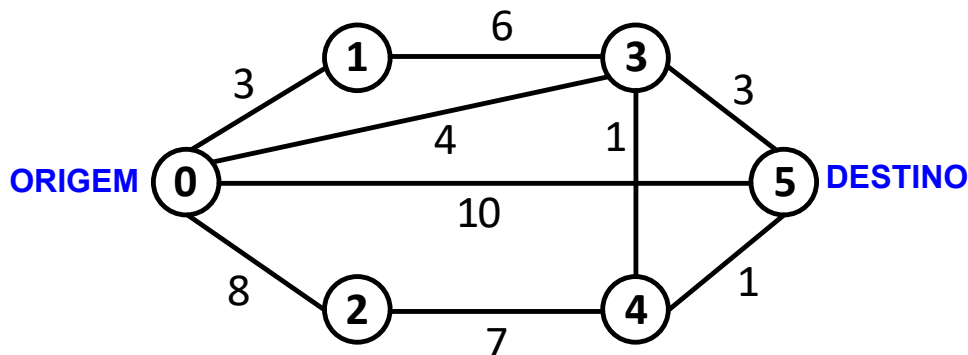
Exemplo

dijkstra($G, v_{\text{Inicio}}, v_{\text{Fim}}$)

```

1 for  $i = 0$  até  $|V|$  do
2    $\text{custo}[i] = \infty$ ;
3    $\text{rota}[i] = v_{\text{Inicio}}$ ;
4 end
5  $\text{custo}[v_{\text{Inicio}}] = 0$ ;
6  $A = V$ ;
7  $F = \emptyset$ ;
8 while  $A \neq \emptyset$  do
9    $v = v \in A$  com menor  $\text{custo}[v]$  entre todos;
10   $F = F \cup \{v\}$ ;
11   $A = A - \{v\}$ ;
12   $N = N - F$ ;
13  for cada  $u \in N$  do
14    if  $\text{custo}[v] + w_{vu} < \text{custo}[u]$  then
15       $\text{custo}[u] = \text{custo}[v] + w_{vu}$ ;
16       $\text{rota}[u] = v$ ;
17    end
18  end
19 end
20  $\text{caminho} = \text{obtemCaminho}(\text{rota}, v_{\text{Inicio}}, v_{\text{Fim}})$ ;
21  $\text{caminho.reverse}$ ;
22 return  $\text{caminho}, \text{custo}[v_{\text{Fim}}]$ 

```



	0	1	2	3	4	5
custo	0	3	8	4	∞	10
rota	-	0	0	0	0	0

$v = 0$ (origem) $F = \{0\}$



4.1. Dijkstra

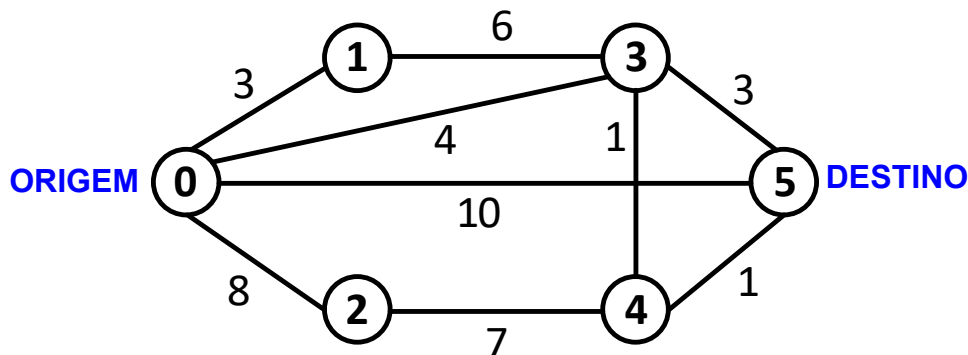
Exemplo

dijkstra($G, v_{\text{Inicio}}, v_{\text{Fim}}$)

```

1 for  $i = 0$  até  $|V|$  do
2    $\text{custo}[i] = \infty$ ;
3    $\text{rota}[i] = v_{\text{Inicio}}$ ;
4 end
5  $\text{custo}[v_{\text{Inicio}}] = 0$ ;
6  $A = V$ ;
7  $F = \emptyset$ ;
8 while  $A \neq \emptyset$  do
9    $v = v \in A$  com menor  $\text{custo}[v]$  entre todos;
10   $F = F \cup \{v\}$ ;
11   $A = A - \{v\}$ ;
12   $N = N - F$ ;
13  for cada  $u \in N$  do
14    if  $\text{custo}[v] + w_{vu} < \text{custo}[u]$  then
15       $\text{custo}[u] = \text{custo}[v] + w_{vu}$ ;
16       $\text{rota}[u] = v$ ;
17    end
18  end
19 end
20  $\text{caminho} = \text{obtemCaminho}(\text{rota}, v_{\text{Inicio}}, v_{\text{Fim}})$ ;
21  $\text{caminho.reverse}$ ;
22 return  $\text{caminho}, \text{custo}[v_{\text{Fim}}]$ 

```



	0	1	2	3	4	5
custo	0	3	8	4	∞	10
rota	-	0	0	0	0	0

$v = 0$ (origem) $F = \{0\}$

$v = 1$ (menor) $F = \{0, 1\}$

**ATUALIZAÇÃO
DOS CUSTOS E
ROTAS**

$\text{custo}[2] = \min(8, 3 + A[1, 2]) = \min(8, \infty) = 8$
 $\text{custo}[3] = \min(4, 3 + A[1, 3]) = \min(4, 9) = 4$
 $\text{custo}[4] = \min(\infty, 3 + A[1, 4]) = \min(\infty, \infty) = \infty$
 $\text{custo}[5] = \min(10, 3 + A[1, 5]) = \min(10, \infty) = 10$



4.1. Dijkstra

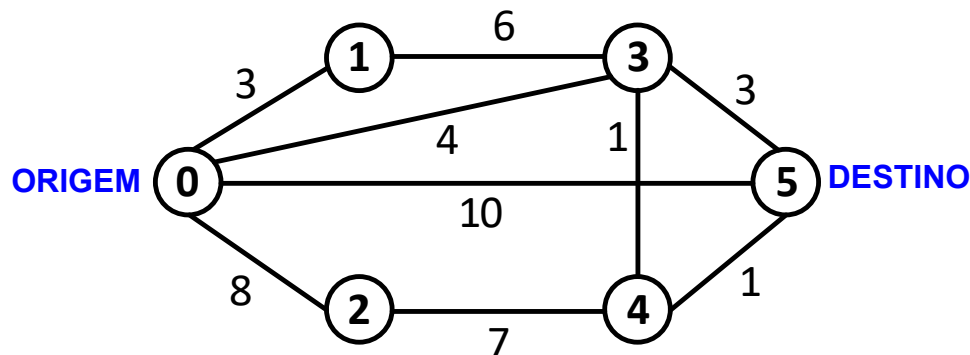
Exemplo

dijkstra(*G*, *vInicio*, *vFim*)

```

1 for i = 0 até |V| do
2   custo[i] = ∞;
3   rota[i] = vInicio;
4 end
5 custo[vInicio] = 0;
6 A = V;
7 F = ∅;
8 while A ≠ ∅ do
9   v = v ∈ A com menor custo[v] entre todos;
10  F = F ∪ {v};
11  A = A − {v};
12  N = N − F;
13  for cada u ∈ N do
14    if custo[v] + wvu < custo[u] then
15      custo[u] = custo[v] + wvu;
16      rota[u] = v;
17    end
18  end
19 end
20 caminho = obterCaminho(rota, vInicio, vFim);
21 caminho.reverse;
22 return caminho, custo[vFim]

```



	0	1	2	3	<u>4</u>	<u>5</u>
custo	0	3	8	4	∞	10
rota	-	0	0	0	0	0

v = 3 (menor) *F* = {0, 1, 3}

custo[2] = min(8, 4 + *A*[3, 2]) = min(8, 4 + ∞) = 8

custo[4] = min(∞, 4 + *A*[3, 4]) = min(∞, 4 + 1) = **5** (atualizar custo/rota)

custo[5] = min(10, 4 + *A*[3, 5]) = min(10, 4 + 3) = **7** (atualizar custo/rota)



4.1. Dijkstra

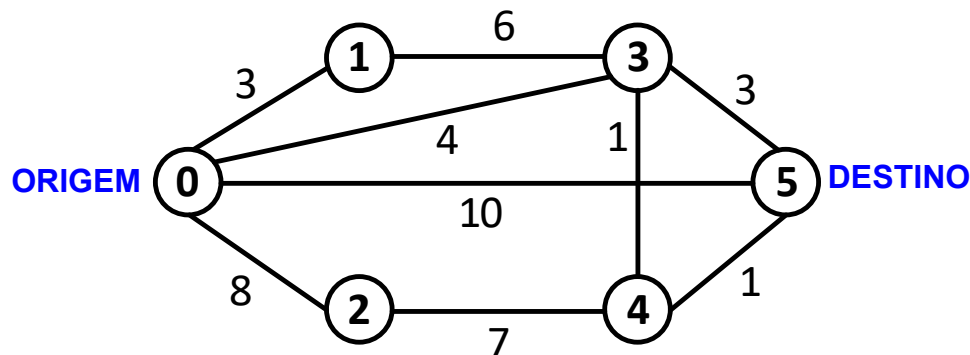
Exemplo

dijkstra(*G*, *vInicio*, *vFim*)

```

1 for i = 0 até |V| do
2   custo[i] = ∞;
3   rota[i] = vInicio;
4 end
5 custo[vInicio] = 0;
6 A = V;
7 F = ∅;
8 while A ≠ ∅ do
9   v = v ∈ A com menor custo[v] entre todos;
10  F = F ∪ {v};
11  A = A - {v};
12  N = N - F;
13  for cada u ∈ N do
14    if custo[v] + wvu < custo[u] then
15      custo[u] = custo[v] + wvu;
16      rota[u] = v;
17    end
18  end
19 end
20 caminho = obterCaminho(rota, vInicio, vFim);
21 caminho.reverse;
22 return caminho, custo[vFim]

```



	0	1	2	3	4	5
custo	0	3	8	4	<u>5</u>	<u>7</u>
rota	-	0	0	0	<u>3</u>	<u>3</u>

$v = 3$ (menor) $F = \{0, 1, 3\}$

$custo[2] = \min(8, 4 + A[3, 2]) = \min(8, 4 + \infty) = 8$

$custo[4] = \min(\infty, 4 + A[3, 4]) = \min(\infty, 4 + 1) = 5$ (atualizar custo/rota)

$custo[5] = \min(10, 4 + A[3, 5]) = \min(10, 4 + 3) = 7$ (atualizar custo/rota)



4.1. Dijkstra

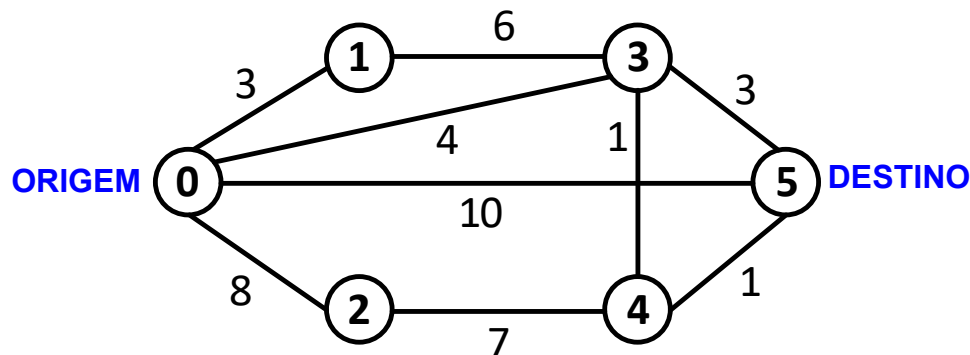
Exemplo

dijkstra($G, v_{\text{Inicio}}, v_{\text{Fim}}$)

```

1 for  $i = 0$  até  $|V|$  do
2    $\text{custo}[i] = \infty$ ;
3    $\text{rota}[i] = v_{\text{Inicio}}$ ;
4 end
5  $\text{custo}[v_{\text{Inicio}}] = 0$ ;
6  $A = V$ ;
7  $F = \emptyset$ ;
8 while  $A \neq \emptyset$  do
9    $v = v \in A$  com menor  $\text{custo}[v]$  entre todos;
10   $F = F \cup \{v\}$ ;
11   $A = A - \{v\}$ ;
12   $N = N - F$ ;
13  for cada  $u \in N$  do
14    if  $\text{custo}[v] + w_{vu} < \text{custo}[u]$  then
15       $\text{custo}[u] = \text{custo}[v] + w_{vu}$ ;
16       $\text{rota}[u] = v$ ;
17    end
18  end
19 end
20  $\text{caminho} = \text{obtemCaminho}(\text{rota}, v_{\text{Inicio}}, v_{\text{Fim}})$ ;
21  $\text{caminho.reverse}$ ;
22 return  $\text{caminho}, \text{custo}[v_{\text{Fim}}]$ 

```



	0	1	2	3	4	<u>5</u>
custo	0	3	8	4	5	7
rota	-	0	0	0	3	3

$v = 3$ (menor) $F = \{0, 1, 3\}$

$\text{custo}[2] = \min(8, 4 + A[3, 2]) = \min(8, 4 + \infty) = 8$

$\text{custo}[4] = \min(\infty, 4 + A[3, 4]) = \min(\infty, 4 + 1) = 5$ (atualizar custo/rota)

$\text{custo}[5] = \min(10, 4 + A[3, 5]) = \min(10, 4 + 3) = 7$ (atualizar custo/rota)

$v = 4$ (menor) $F = \{0, 1, 3, 4\}$

$\text{custo}[2] = \min(8, 5 + A[4, 2]) = \min(8, 5 + 7) = 8$

$\text{custo}[5] = \min(7, 5 + A[4, 5]) = \min(7, 5 + 1) = 6$ (atualizar custo/rota)



4.1. Dijkstra

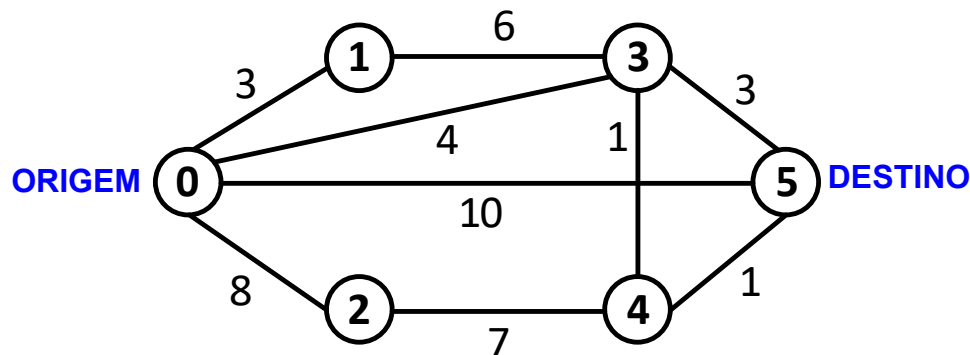
Exemplo

dijkstra($G, v_{\text{Inicio}}, v_{\text{Fim}}$)

```

1 for  $i = 0$  até  $|V|$  do
2    $\text{custo}[i] = \infty$ ;
3    $\text{rota}[i] = v_{\text{Inicio}}$ ;
4 end
5  $\text{custo}[v_{\text{Inicio}}] = 0$ ;
6  $A = V$ ;
7  $F = \emptyset$ ;
8 while  $A \neq \emptyset$  do
9    $v = v \in A$  com menor  $\text{custo}[v]$  entre todos;
10   $F = F \cup \{v\}$ ;
11   $A = A - \{v\}$ ;
12   $N = N - F$ ;
13  for cada  $u \in N$  do
14    if  $\text{custo}[v] + w_{vu} < \text{custo}[u]$  then
15       $\text{custo}[u] = \text{custo}[v] + w_{vu}$ ;
16       $\text{rota}[u] = v$ ;
17    end
18  end
19 end
20  $\text{caminho} = \text{obtemCaminho}(\text{rota}, v_{\text{Inicio}}, v_{\text{Fim}})$ ;
21  $\text{caminho.reverse}$ ;
22 return  $\text{caminho}, \text{custo}[v_{\text{Fim}}]$ 

```



	0	1	2	3	4	5
custo	0	3	8	4	5	<u>6</u>
rota	-	0	0	0	3	<u>4</u>

$v = 3$ (menor) $F = \{0, 1, 3\}$

$\text{custo}[2] = \min(8, 4 + A[3, 2]) = \min(8, 4 + \infty) = 8$

$\text{custo}[4] = \min(\infty, 4 + A[3, 4]) = \min(\infty, 4 + 1) = 5$ (atualizar custo/rota)

$\text{custo}[5] = \min(10, 4 + A[3, 5]) = \min(10, 4 + 3) = 7$ (atualizar custo/rota)

$v = 4$ (menor) $F = \{0, 1, 3, 4\}$

$\text{custo}[2] = \min(8, 5 + A[4, 2]) = \min(8, 5 + 7) = 8$

$\text{custo}[5] = \min(7, 5 + A[4, 5]) = \min(7, 5 + 1) = 6$ (atualizar custo/rota)



4.1. Dijkstra

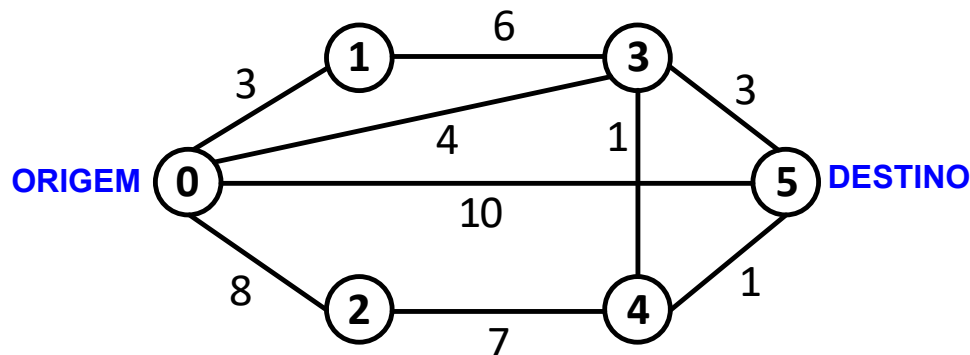
Exemplo

dijkstra(*G*, *vInicio*, *vFim*)

```

1 for i = 0 até |V| do
2   | custo[i] = ∞;
3   | rota[i] = vInicio;
4 end
5 custo[vInicio] = 0;
6 A = V;
7 F = ∅;
8 while A ≠ ∅ do
9   | v = v ∈ A com menor custo[v] entre todos;
10  | F = F ∪ {v};
11  | A = A − {v};
12  | N = N − F;
13  | for cada u ∈ N do
14    | if custo[v] + wvu < custo[u] then
15      | | custo[u] = custo[v] + wvu;
16      | | rota[u] = v;
17    | end
18  | end
19 end
20 caminho = obtemCaminho(rota, vInicio, vFim);
21 caminho.reverse;
22 return caminho, custo[vFim]

```



	0	1	2	3	4	5
custo	0	3	8	4	5	6
rota	-	0	0	0	3	4

v = 5 (menor) *F* = {0, 1, 3, 4, 5}

custo[2] = *min*(8, 6 + ∞) = 8

Para obter o caminho mínimo **inicie pelo vértice destino e percorra a rota dos precedentes até alcançar a origem** (5 → 4 → 3 → 0).

O **caminho mínimo** entre 0 e 5 é: 0, 3, 4, 5.



4.1. Dijkstra

Comentários

- O algoritmo de Dijkstra possui complexidade $O(n^2)$.

CRÍTICAS AO ALGORITMO

- O algoritmo é **incapaz de calcular** os caminhos mínimos caso existam arestas com **custo negativo**.
- O algoritmo só calcula os caminhos mínimos a partir de uma **única origem**.



Comentários

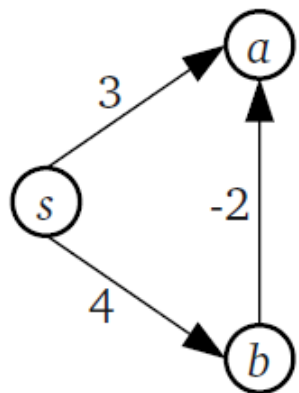
- O algoritmo de Dijkstra possui complexidade $O(n^2)$.

CRÍTICAS AO ALGORITMO

- O algoritmo é incapaz de calcular os caminhos mínimos caso existam arestas com custo negativo.
- O algoritmo só calcula os caminhos mínimos a partir de uma única origem.
- Para calcular os caminhos mínimos de todos os vértices para todos os vértices, o algoritmo deve ser executado uma vez para cada vértice do grafo, com complexidade total $O(n^3)$ na implementação simples.
- Se for utilizada uma estrutura *Heap* e listas de adjacências na representação do grafo a complexidade é reduzida para $O((V + E) \log E)$ pois determinar o menor elemento e atualizar a *Heap* pode ser feito em tempo logarítmico.
- Melhor implementação (ano 2000) possui complexidade $O(n \log \log m)$.



Limitação



- Uma limitação do algoritmo é que as instâncias devem obedecer à **desigualdade triangular** (um dos lados do triângulo é menor ou igual a soma dos outros dois lados):

$$l_{ij} \leq l_{ik} + l_{kj} \quad \forall i, j, k \in G$$

- O algoritmo considera que qualquer caminho de s para outro vértice deve passar apenas por vértices mais próximos de s .
- No exemplo, o caminho mínimo entre s e a passa por b , mas o arco incidente em b possui maior custo que o arco que incide diretamente em a :

$$(s, a) = 3, \text{ sendo que } (s, b) + (b, a) = 4 - 2 = 2$$



4. Caminho Mínimo

Algoritmo de Bellman-Ford

Arestas de peso negativo: Além das distâncias geográficas, caminhos mínimos podem modelar situações reais que necessitam de arestas com pesos negativos.

- Movimentações financeiras, nas quais pode ocorrer a obtenção de lucro ou prejuízo, principalmente quando em operações de câmbio;
- Um taxista que recebe mais dinheiro do que gasta com combustível a cada viagem: se o táxi roda vazio, ele gasta mais do que recebe;
- Um entregador que necessita atravessar um pedágio e pode acabar pagando mais do que recebe para entregar encomendas;
- A energia gerada e consumida durante uma reação química.



4. Caminho Mínimo

Algoritmo de Bellman-Ford

Arestas de peso negativo: Além das distâncias geográficas, caminhos mínimos podem modelar situações reais que necessitam de arestas com pesos negativos.

- Movimentações financeiras, nas quais pode ocorrer a obtenção de lucro ou prejuízo, principalmente quando em operações de câmbio;
- Um taxista que recebe mais dinheiro do que gasta com combustível a cada viagem: se o táxi roda vazio, ele gasta mais do que recebe;
- Um entregador que necessita atravessar um pedágio e pode acabar pagando mais do que recebe para entregar encomendas;
- A energia gerada e consumida durante uma reação química.

Alguns autores denominam o algoritmo de Ford-Moore-Bellman, em homenagem a outros três autores que propuseram o mesmo algoritmo em anos diferentes: Lester Ford (1956), Edward Moore (1957), Richard Bellman (1958).



4.2. Bellman-Ford

Princípio de Funcionamento

- Calcula caminhos mais curtos via *programação dinâmica bottom-up*.
- Ao invés de fechar um vértice por iteração como faz Dijkstra, *examina todos os vértices* de um grafo orientado por iteração *até que atualizações não sejam mais possíveis*.
- Em um grafo com n vértices, qualquer caminho possui no máximo $n-1$ arestas, portanto, cada vértice é examinado no máximo $n-1$ vezes. Com esta estratégia, *é possível calcular caminhos mínimos em grafos com arestas de peso negativo*.



4.2. Bellman-Ford

Princípio de Funcionamento

- Calcula caminhos mais curtos via programação dinâmica *bottom-up*.
- Ao invés de fechar um vértice por iteração, como faz Dijkstra, examina todos os vértices de um grafo orientado por iteração até que atualizações não sejam mais possíveis.
- Em um grafo com n vértices, qualquer caminho possui no máximo $n-1$ arestas, portanto, cada vértice é examinado no máximo $n-1$ vezes. Com esta estratégia, é possível calcular caminhos mínimos em grafos com arestas de peso negativo.
- Assim como o algoritmo de Dijkstra, baseia-se no **princípio de relaxação**: uma aproximação da **distância da origem até cada vértice é gradualmente atualizada** por valores mais baixos até que a solução seja obtida.
- Se, em alguma iteração do algoritmo os caminhos até cada um dos vértices permanecerem inalterados, não haverá atualizações nas próximas iterações e o algoritmo pode terminar.
- Entretanto, se houver atualizações na última iteração do algoritmo, é sinal de que há pelo menos um ciclo negativo no grafo, dado que algum caminho terá n arestas ou mais.



4.2. Bellman-Ford

Algoritmo

bellmanFord(*G*, *vInicio*, *vFim*)

```
1 for i = 0 até |V| do
2   | custo[i] = ∞;
3   | rota[i] = vInicio;
4 end
5 custo[vInicio] = 0;
6 for i = 0 até |V| do
7   for cada aresta (v, u) ∈ E do
8     if custo[u] > custo[v] + wvu then
9       | custo[u] = custo[v] + wvu;
10      | rota[u] = v;
11    end
12  end
13 end
14 for cada aresta (v, u) ∈ E do
15   if custo[u] > custo[v] + wvu then
16     | return False
17   end
18 end
19 caminho = obterCaminho(rota, vInicio, vFim);
20 caminho.reverse;
21 return caminho, custo[vFim]
```

ESTRUTURAS

- *custo*[*v*] : vetor que armazena o custo (distância) entre o vértice origem e o vértice *v*.
- *rota*[*v*] : vetor que armazena o índice do vértice que precede o vértice *v*, no caminho cuja distância consta no vetor *custo*[*v*].
- *w_{vu}* : valor do peso da aresta ou arco que conecta *v* e *u*.



4.2. Bellman-Ford

Algoritmo

bellmanFord(*G*, *vInicio*, *vFim*)

```
1 for i = 0 até |V| do
2   custo[i] = ∞;
3   rota[i] = vInicio;
4 end
5 custo[vInicio] = 0;
6 for i = 0 até |V| do
7   for cada aresta (v, u) ∈ E do
8     if custo[u] > custo[v] + wvu then
9       custo[u] = custo[v] + wvu;
10      rota[u] = v;
11    end
12  end
13 end
14 for cada aresta (v, u) ∈ E do
15   if custo[u] > custo[v] + wvu then
16     return False
17   end
18 end
19 caminho = obtenCaminho(rota, vInicio, vFim);
20 caminho.reverse;
21 return caminho, custo[vFim]
```

ESTRUTURAS

- *custo*[*v*] : vetor que armazena o custo (distância) entre o vértice origem e o vértice *v*.
- *rota*[*v*] : vetor que armazena o índice do vértice que precede o vértice *v*, no caminho cuja distância consta no vetor *custo*[*v*].
- *w_{vu}* : valor do peso da aresta ou arco que conecta *v* e *u*.

Os **vetores** de custo e rota **são inicializados** considerando o vértice de id 0 como origem (linhas 1 a 5).

Considerando o caminho de *s* para cada vértice de *G*. Para cada aresta (*v*, *u*):

Se o custo de adicionar o vértice *u* no caminho for maior que o custo do vértice *v* mais o peso da aresta que conecta os vértices *v* e *u* (linha 8).

Atualiza o custo do vértice *u* para o custo do vértice *v* mais o peso da aresta (*v*, *u*).

Atualiza a rota do vértice *u* considerando o vértice *v* como precedente.



4.2. Bellman-Ford

Algoritmo

bellmanFord(*G*, *vInicio*, *vFim*)

```
1 for i = 0 até |V| do
2   custo[i] = ∞;
3   rota[i] = vInicio;
4 end
5 custo[vInicio] = 0;
6 for i = 0 até |V| do
7   for cada aresta (v, u) ∈ E do
8     if custo[u] > custo[v] + wvu then
9       custo[u] = custo[v] + wvu;
10      rota[u] = v;
11    end
12  end
13 end
14 for cada aresta (v, u) ∈ E do
15   if custo[u] > custo[v] + wvu then
16     return False
17   end
18 end
19 caminho = obterCaminho(rota, vInicio, vFim);
20 caminho.reverse;
21 return caminho, custo[vFim]
```

ESTRUTURAS

- *custo*[*v*] : vetor que armazena o custo (distância) entre o vértice origem e o vértice *v*.
- *rota*[*v*] : vetor que armazena o índice do vértice que precede o vértice *v*, no caminho cuja distância consta no vetor *custo*[*v*].
- *w_{vu}* : valor do peso da aresta ou arco que conecta *v* e *u*.

	0	1	2	3	4
custo	0	∞	∞	∞	∞
rota	-	0	0	0	0

Para cada aresta (*v*, *u*) **verifica se o grafo possui um ciclo de peso negativo** que seja acessível a partir do vértice origem.

Caso exista ao menos um ciclo negativo o algoritmo retorna False (linhas 14 a 18).

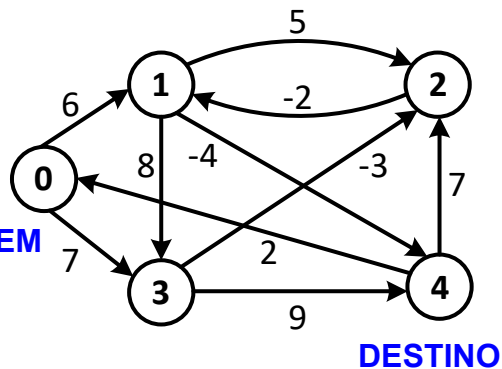
Caso não exista nenhum ciclo negativo o algoritmo obtém o caminho e o retorna junto com o custo (linhas 19 a 21).



4.2. Bellman-Ford

Exemplo

$E = (0,1), (0,3), (1,2), (1,3), (1,4), (2,1), (3,2), (3,4), (4,0), (4,2)$



	0	1	2	3	4
custo	0	∞	∞	∞	∞
rota	-	0	0	0	0

bellmanFord(G, vInicio, vFim)

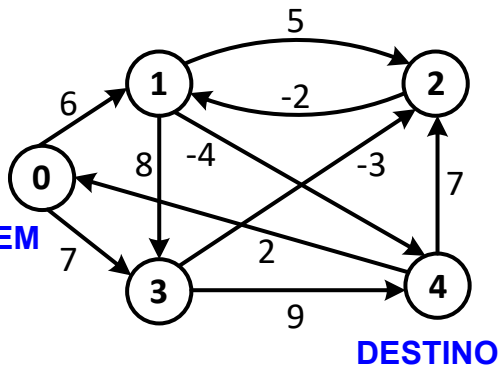
```

1 for i = 0 até |V| do
2   custo[i] =  $\infty$ ;
3   rota[i] = vInicio;
4 end
5 custo[vInicio] = 0;
6 for i = 0 até |V| do
7   for cada aresta (v,u) ∈ E do
8     if custo[u] > custo[v] + wvu then
9       custo[u] = custo[v] + wvu;
10      rota[u] = v;
11    end
12  end
13 end
14 for cada aresta (v,u) ∈ E do
15   if custo[u] > custo[v] + wvu then
16     return False
17   end
18 end
19 caminho = obterCaminho(rota, vInicio, vFim);
20 caminho.reverse;
21 return caminho, custo[vFim]
  
```



4.2. Bellman-Ford

Exemplo



bellmanFord(G, vInicio, vFim)

```

1 for i = 0 até |V| do
2   custo[i] = ∞;
3   rota[i] = vInicio;
4 end
5 custo[vInicio] = 0;
6 for i = 0 até |V| do
7   for cada aresta (v, u) ∈ E do
8     if custo[u] > custo[v] + wvu then
9       custo[u] = custo[v] + wvu;
10      rota[u] = v;
11    end
12  end
13 end
14 for cada aresta (v, u) ∈ E do
15   if custo[u] > custo[v] + wvu then
16     return False
17   end
18 end
19 caminho = obtemCaminho(rota, vInicio, vFim);
20 caminho.reverse;
21 return caminho, custo[vFim]
```

$E = (0,1), (0,3), (1,2), (1,3), (1,4), (2,1), (3,2), (3,4), (4,0), (4,2)$

PASSO 1

	0	1	2	3	4
custo	0	6	∞	7	∞
rota	0	0	0	0	0

$v = 0$

$\text{custo}[1] = \min(\infty, 0 + 6) = 6$ (atualiza custo e rota)

$\text{custo}[3] = \min(\infty, 0 + 7) = 7$ (atualiza custo e rota)

	0	1	2	3	4
custo	0	6	11	7	2
rota	0	0	1	0	1

$v = 1$

$\text{custo}[2] = \min(\infty, 6 + 5) = 11$ (atualiza custo e rota)

$\text{custo}[3] = \min(7, 6 + 8) = 7$

$\text{custo}[4] = \min(\infty, 6 + (-4)) = 2$ (atualiza custo e rota)

	0	1	2	3	4
custo	0	6	11	7	2
rota	0	0	1	0	1

$v = 2$

$\text{custo}[1] = \min(6, 11 + (-2)) = 6$

	0	1	2	3	4
custo	0	6	4	7	2
rota	0	0	3	0	1

$v = 3$

$\text{custo}[2] = \min(11, 7 + (-3)) = 4$ (atualiza custo e rota)

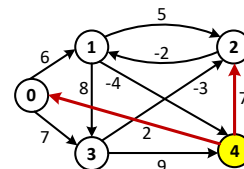
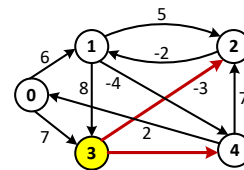
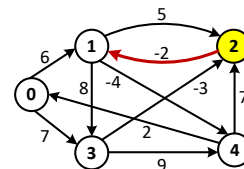
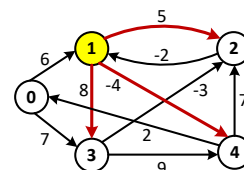
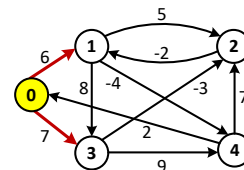
$\text{custo}[4] = \min(2, 7 + 9) = 2$

	0	1	2	3	4
custo	0	6	4	7	2
rota	0	0	3	0	1

$v = 4$

$\text{custo}[0] = \min(0, 2 + 2) = 0$

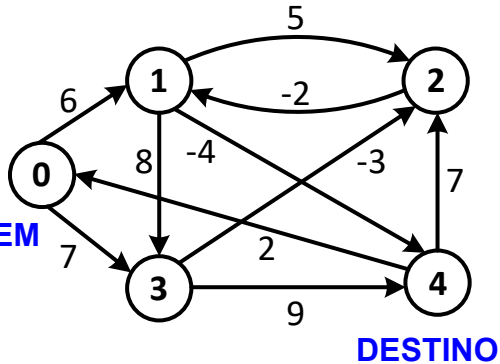
$\text{custo}[2] = \min(4, 2 + 7) = 4$





4.2. Bellman-Ford

Exemplo



$E = (0,1), (0,3), (1,2), (1,3), (1,4), (2,1), (3,2), (3,4), (4,0), (4,2)$

bellmanFord(G, vInicio, vFim)

```

1 for i = 0 até |V| do
2   custo[i] = ∞;
3   rota[i] = vInicio;
4 end
5 custo[vInicio] = 0;
6 for i = 0 até |V| do
7   for cada aresta (v, u) ∈ E do
8     if custo[u] > custo[v] + wvu then
9       custo[u] = custo[v] + wvu;
10      rota[u] = v;
11    end
12  end
13 end
14 for cada aresta (v, u) ∈ E do
15   if custo[u] > custo[v] + wvu then
16     return False
17   end
18 end
19 caminho = obtemCaminho(rota, vInicio, vFim);
20 caminho.reverse;
21 return caminho, custo[vFim]
```

PASSO 1

	0	1	2	3	4
custo	0	6	∞	7	∞
rota	0	0	0	0	0

$v = 0$

custo[1] = $\min(\infty, 0 + 6) = 6$ (atualiza custo e rota)
 custo[3] = $\min(\infty, 0 + 7) = 7$ (atualiza custo e rota)

	0	1	2	3	4
custo	0	6	11	7	2
rota	0	0	1	0	1

$v = 1$

custo[2] = $\min(\infty, 6 + 5) = 11$ (atualiza custo e rota)
 custo[3] = $\min(7, 6 + 8) = 7$
 custo[4] = $\min(\infty, 6 + (-4)) = 2$ (atualiza custo e rota)

	0	1	2	3	4
custo	0	6	11	7	2
rota	0	0	1	0	1

$v = 2$

custo[1] = $\min(6, 11 + (-2)) = 6$

	0	1	2	3	4
custo	0	6	4	7	2
rota	0	0	3	0	1

$v = 3$

custo[2] = $\min(11, 7 + (-3)) = 4$ (atualiza custo e rota)
 custo[4] = $\min(2, 7 + 9) = 2$

	0	1	2	3	4
custo	0	6	4	7	2
rota	0	0	3	0	1

$v = 4$

custo[0] = $\min(0, 2 + 2) = 0$
 custo[2] = $\min(4, 2 + 7) = 4$

PASSO 2

	0	1	2	3	4
custo	0	6	4	7	2
rota	0	0	3	0	1

$v = 0$

custo[1] = $\min(6, 0 + 6) = 6$
 custo[3] = $\min(7, 0 + 7) = 7$

	0	1	2	3	4
custo	0	6	4	7	2
rota	0	0	3	0	1

$v = 1$

custo[2] = $\min(4, 6 + 5) = 4$
 custo[3] = $\min(7, 6 + 8) = 7$
 custo[4] = $\min(2, 6 + (-4)) = 2$

	0	1	2	3	4
custo	0	2	4	7	2
rota	0	2	3	0	1

$v = 2$

custo[1] = $\min(6, 4 + (-2)) = 2$ (atualiza custo e rota)

	0	1	2	3	4
custo	0	2	4	7	2
rota	0	2	3	0	1

$v = 3$

custo[2] = $\min(4, 7 + (-3)) = 4$
 custo[4] = $\min(2, 7 + 9) = 2$

	0	1	2	3	4
custo	0	2	4	7	2
rota	0	2	3	0	1

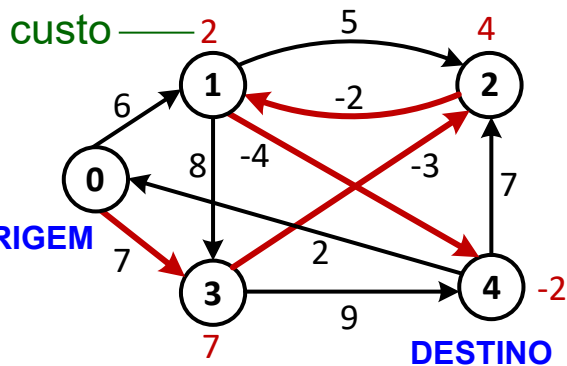
$v = 4$

custo[0] = $\min(0, 2 + 2) = 0$
 custo[2] = $\min(4, 2 + 7) = 4$



4.2. Bellman-Ford

Exemplo



$E = (0,1), (0,3), (1,2), (1,3), (1,4), (2,1), (3,2), (3,4), (4,0), (4,2)$

PASSO 3

	0	1	2	3	4
custo	0	2	4	7	2
rota	0	2	3	0	1

$v = 0$

$\text{custo}[1] = \min(2, 0 + 6) = 2$

$\text{custo}[3] = \min(7, 0 + 7) = 7$

	0	1	2	3	4
custo	0	2	4	7	-2
rota	0	2	3	0	1

$v = 1$

$\text{custo}[2] = \min(4, 2 + 5) = 4$

$\text{custo}[3] = \min(7, 2 + 8) = 7$

$\text{custo}[4] = \min(2, 2 + (-4)) = -2$ (atualiza custo e rota)

	0	1	2	3	4
custo	0	2	4	7	-2
rota	0	2	3	0	1

$v = 2$

$\text{custo}[1] = \min(2, 4 + (-2)) = 2$

	0	1	2	3	4
custo	0	2	4	7	-2
rota	0	2	3	0	1

$v = 3$

$\text{custo}[2] = \min(4, 7 + (-3)) = 4$

$\text{custo}[4] = \min(-2, 7 + 9) = -2$

	0	1	2	3	4
custo	0	2	4	7	-2
rota	0	2	3	0	1

$v = 4$

$\text{custo}[0] = \min(0, -2 + 2) = 0$

$\text{custo}[2] = \min(4, -2 + 7) = 4$

PASSO 4

	0	1	2	3	4
custo	0	2	4	7	-2
rota	0	2	3	0	1

$v = 0$

$\text{custo}[1] = \min(2, 0 + 6) = 2$

$\text{custo}[3] = \min(7, 0 + 7) = 7$

	0	1	2	3	4
custo	0	2	4	7	-2
rota	0	2	3	0	1

$v = 1$

$\text{custo}[2] = \min(4, 2 + 5) = 4$

$\text{custo}[3] = \min(7, 2 + 8) = 7$

$\text{custo}[4] = \min(-2, 2 + (-4)) = -2$

	0	1	2	3	4
custo	0	2	4	7	-2
rota	0	2	3	0	1

$v = 2$

$\text{custo}[1] = \min(2, 4 + (-2)) = 2$

	0	1	2	3	4
custo	0	2	4	7	-2
rota	0	2	3	0	1

$v = 3$

$\text{custo}[2] = \min(4, 7 + (-3)) = 4$

$\text{custo}[4] = \min(-2, 7 + 9) = -2$

	0	1	2	3	4
custo	0	2	4	7	-2
rota	0	2	3	0	1

$v = 4$

$\text{custo}[0] = \min(0, -2 + 2) = 0$

$\text{custo}[2] = \min(4, -2 + 7) = 4$

bellmanFord(G, vInicio, vFim)

```

1 for i = 0 até |V| do
2   custo[i] = ∞;
3   rota[i] = vInicio;
4 end
5 custo[vInicio] = 0;
6 for i = 0 até |V| do
7   for cada aresta (v, u) ∈ E do
8     if custo[u] > custo[v] + wvu then
9       custo[u] = custo[v] + wvu;
10      rota[u] = v;
11    end
12  end
13 end
14 for cada aresta (v, u) ∈ E do
15   if custo[u] > custo[v] + wvu then
16     return False
17   end
18 end
19 caminho = obtémCaminho(rota, vInicio, vFim);
20 caminho.reverse;
21 return caminho, custo[vFim]
```



Comentários

O algoritmo de Bellman-Ford possui complexidade $O(n^3)$.

Uma versão proposta por Yen (1970) possui complexidade $O(nm)$ no pior caso:

- Se $custo[v]$ não se alterar desde a última vez que seus antecessores foram analisados, então não é necessário examinar novamente seus arcos de saída.
- O comprimento do laço externo é reduzido de $n-1$ para $n/2$ através de uma ordenação linear dos vértices e posterior partição dos mesmos.
- Yen, Jin Y. (1970). "An algorithm for finding shortest routes from all source nodes to a given destination in general networks". Quarterly of Applied Mathematics 27: 526–530.



4. Caminho Mínimo

Algoritmo de Floyd-Warshall

O algoritmo proposto por Robert Floyd em 1962 é baseado no algoritmo de Stephen Warshall do mesmo ano para cálculo de fechos transitivos em grafos.

- Warshall, Stephen (January 1962). “*A theorem on Boolean matrices*”. *Journal of the ACM* 9 (1): 11–12. doi:10.1145/321105.321107.
- Floyd, Robert W. (June 1962). “*Algorithm 97: Shortest Path*”. *Communications of the ACM* 5 (6): 345. doi:10.1145/367766.368168.
- A versão do algoritmo com três laços aninhados é devida a Peter Ingerman. Ingerman, Peter Z. (November 1962). “*Algorithm 141: Path Matrix*”. *Communications of the ACM* 5 (11): 556. doi:10.1145/368996.369016



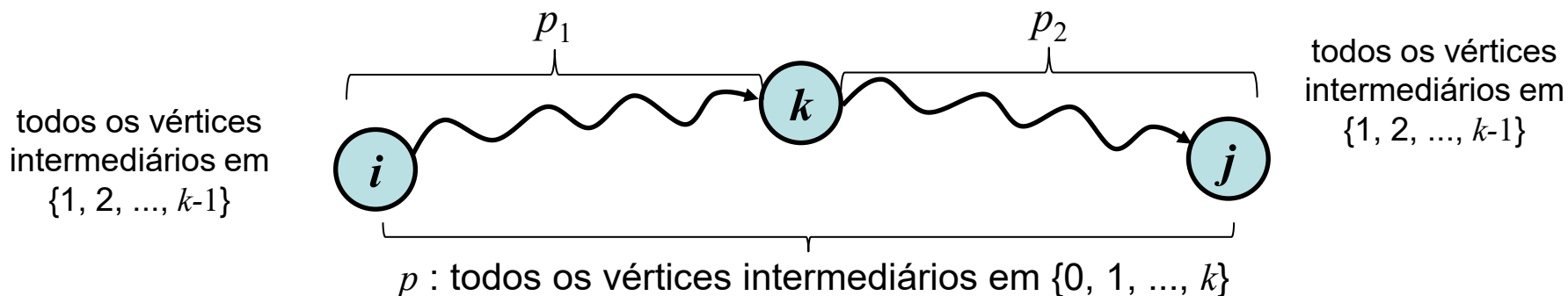
4. Caminho Mínimo

Algoritmo de Floyd-Warshall

O algoritmo proposto por Robert Floyd em 1962 é baseado no algoritmo de Stephen Warshall do mesmo ano para cálculo de fechos transitivos em grafos.

- Warshall, Stephen (January 1962). "A theorem on Boolean matrices". Journal of the ACM 9 (1): 11–12. doi:10.1145/321105.321107.
- Floyd, Robert W. (June 1962). "Algorithm 97: Shortest Path". Communications of the ACM 5 (6): 345. doi:10.1145/367766.368168.
- A versão do algoritmo com três laços aninhados é devida a Peter Ingerman. Ingerman, Peter Z. (November 1962). "Algorithm 141: Path Matrix". Communications of the ACM 5 (11): 556. doi:10.1145/368996.369016

Na caracterização da estrutura de um caminho mínimo, considera os **vértices intermediários** de um caminho com origem em v_0 e destino v_ℓ ou $\{v_1, v_2, \dots, v_{\ell-1}\}$.





4.3. Floyd-Warshall

Princípio de Funcionamento

- Algoritmo de **programação dinâmica** *bottom-up*.
- Calcula os caminhos mais curtos entre todos os pares de vértices de um **grafo direcionado** e **ponderado** (permite pesos negativos), mas **que não possua ciclos de custo negativo**.
- Também de maneira similar a Dijkstra, usa o **conceito de relaxação do comprimento dos caminhos mais curtos**: incrementalmente, aprimora-se uma estimativa utilizada, até que o valor ótimo seja atingido.



4.3. Floyd-Warshall

Princípio de Funcionamento

- Algoritmo de programação dinâmica *bottom-up*.
- Calcula os caminhos mais curtos entre todos os pares de vértices de um grafo direcionado e ponderado (permite pesos negativos), mas que não possua ciclos de custo negativo.
- Também de maneira similar a Dijkstra, usa o conceito de relaxação do comprimento dos caminhos mais curtos: incrementalmente, aprimora-se uma estimativa utilizada, até que o valor ótimo seja atingido.
- O algoritmo compara os caminhos entre dois vértices v e u passando por k vértices intermediários, $k = 1, \dots, n-1$.
- Todos os caminhos entre cada par de vértices são analisados, sendo que uma matriz armazena o valor dos caminhos mais curtos entre os vértices, porém, não há informação sobre a composição do caminho.

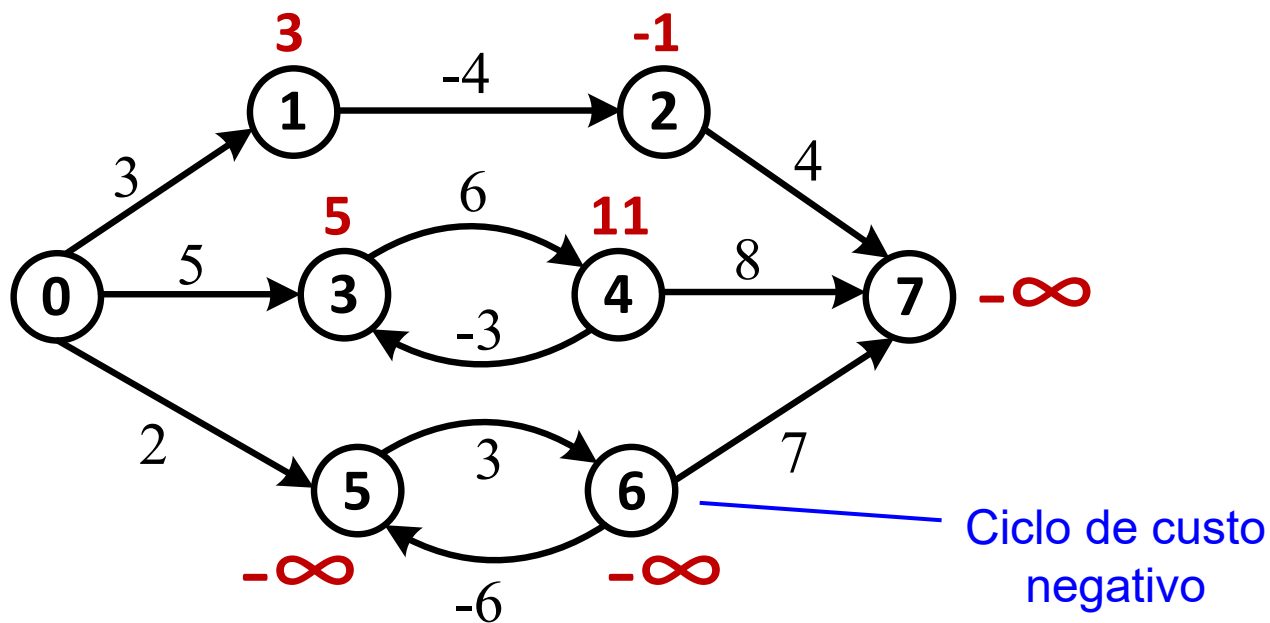


4.3. Floyd-Warshall

Ciclos de Custo Negativo

O algoritmo **detecta ciclos de custo negativo**. Caso haja valores negativos na diagonal principal da matriz de pesos W (mesmo durante a execução do algoritmo), significa que o vértice está contido em um ciclo de custo negativo.

Algumas versões do algoritmo consideram que não haverá ciclos de custo negativo e definem inicialmente a distância de um vértice para si próprio como $-\infty$.





4.3. Floyd-Warshall

Algoritmo

floydWarshall(W)

```

1  n = quantidade de linhas de W;
2  D(0) = W;
3  for k = 0 até n do
4      for v = 0 até n do
5          for u = 0 até n do
6              | dvu(k) = menor(dvu(k-1), dvk(k-1) + dku(k-1));
7              end
8          end
9  end
10 return D(n)

```

$$d_{vu}^{(k)} = \begin{cases} w_{vu} & \text{se } k = 0 \\ \text{menor}(d_{vu}^{(k-1)}, d_{vk}^{(k-1)} + d_{ku}^{(k-1)}) & \text{se } k \geq 1 \end{cases}$$

ESTRUTURAS

- W : Matriz de adjacências $n \times n$ com custos das arestas.
 - $D^{(0)}$: Matriz de distâncias de índice 0.
 - $d_{ij}^{(k)}$: Célula da matriz $D^{(k)}$ com peso da aresta ou arco que conecta os vértices v e u .
-

O algoritmo **tem como entrada uma matriz de custos W** definida conforme [equação](#).

Inicia obtendo a quantidade de linhas da matriz com custos W (linha 1).

Define a matriz $D^{(0)}$ como cópia de W (linha 2).

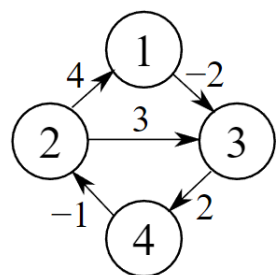
Para cada vértice intermediário, calcular os valores $d_{vu}^{(k)}$ em ordem de valores crescentes de k (linhas 3 a 9)

Retorna a matriz de pesos $D^{(n)}$ (linha 10).

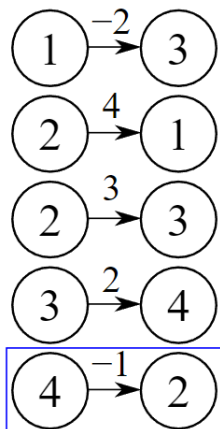


4.3. Floyd-Warshall

Exemplo 1



$k = 0$:



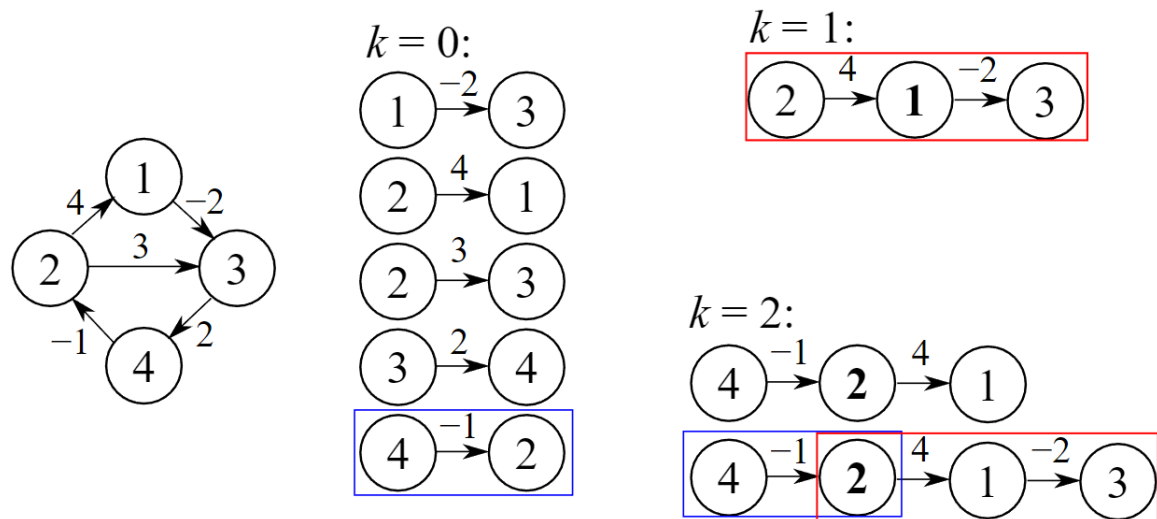
Iteração $k = 0$:

Somente os caminhos representados por uma única adjacência no grafo são conhecidos.



4.3. Floyd-Warshall

Exemplo 1



Iteração $k = 1$:

Todos os caminhos que passam pelo vértice 1 são descobertos. Em particular, o caminho [2,1,3] substitui o caminho [2, 3].

Iteração $k = 2$:

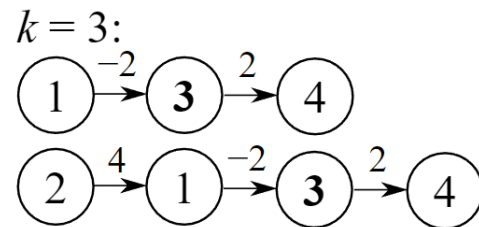
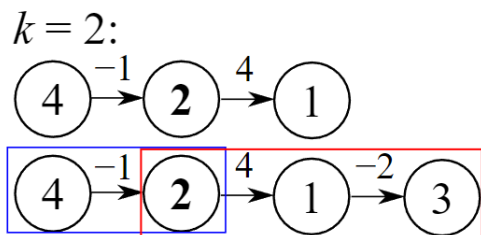
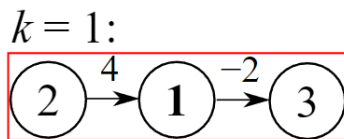
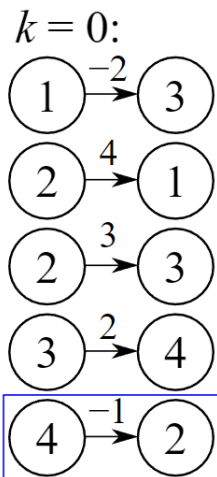
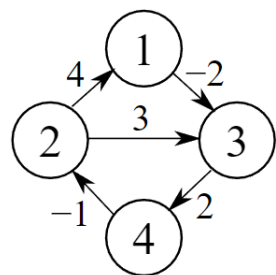
Todos os caminhos que passam pelo vértice 2 ou pelos vértices 2 e 1 são descobertos.

Em particular, o caminho [4, 2, 3] não é considerado, dado que [4, 2, 1, 3] é um caminho mais curto até então.



4.3. Floyd-Warshall

Exemplo 1



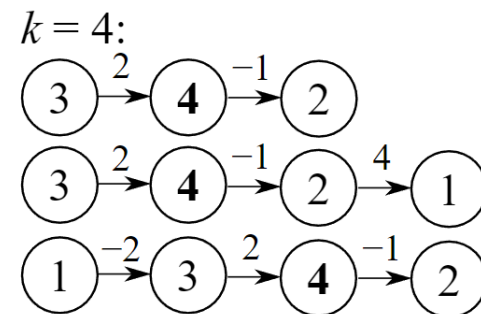
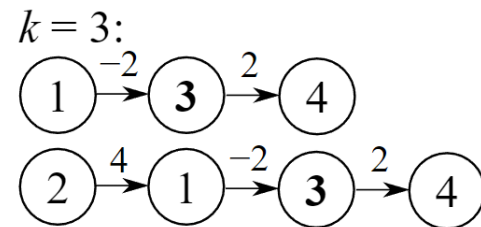
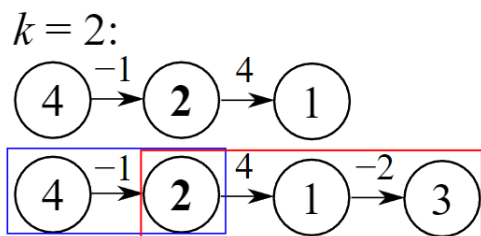
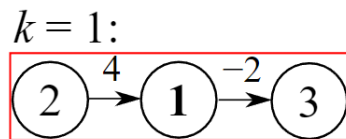
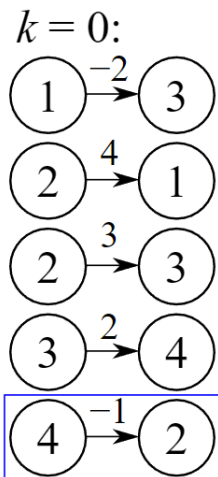
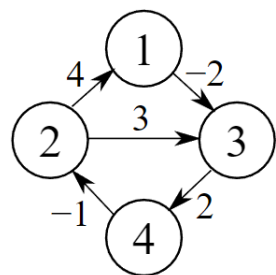
Iteração $k = 3$:

Todos os caminhos que passam pelo vértice 3 ou pelos vértices 3 e 2 ou 1 são descobertos.



4.3. Floyd-Warshall

Exemplo 1



Iteração $k = 3$:

Todos os caminhos que passam pelo vértice 3 ou pelos vértices 3 e 2 ou 1 são descobertos.

Iteração $k = 4$:

Por fim, nesta iteração todos os caminhos mínimos são determinados.



4.3. Floyd-Warshall

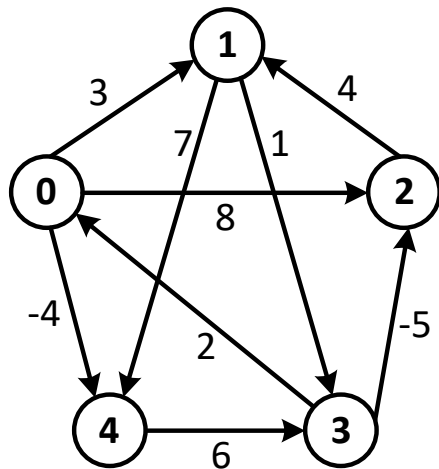
Exemplo 2

floydWarshall(W)

```

1  n = quantidade de linhas de W;
2   $D^{(0)} = W$ ;
3  for k = 0 até n do
4      for v = 0 até n do
5          for u = 0 até n do
6               $d_{vu}^{(k)} = \text{menor}(d_{vu}^{(k-1)}, d_{vk}^{(k-1)} + d_{ku}^{(k-1)});$ 
7          end
8      end
9  end
10 return  $D^{(n)}$ 
```

$$D^{(0)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \end{matrix}$$





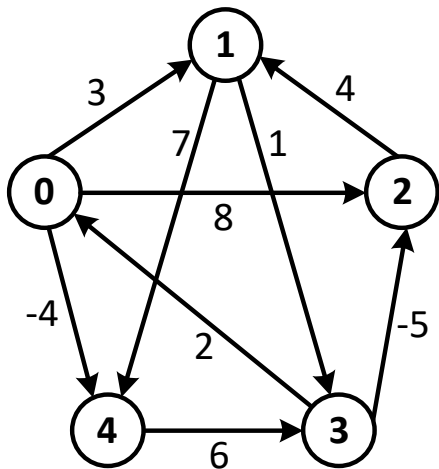
4.3. Floyd-Warshall

Exemplo 2

```
floydWarshall(W)  
1  $n =$  quantidade de linhas de  $W$ ;  
2  $D^{(0)} = W$ ;  
3 for  $k = 0$  até  $n$  do  
4   for  $v = 0$  até  $n$  do  
5     for  $u = 0$  até  $n$  do  
6        $d_{vu}^{(k)} = \text{menor}(d_{vu}^{(k-1)}, d_{vk}^{(k-1)} + d_{ku}^{(k-1)})$ ;  
7     end  
8   end  
9 end  
10 return  $D^{(n)}$ 
```

$$D^{(0)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \end{matrix}$$

$$D^{(1)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \end{matrix}$$





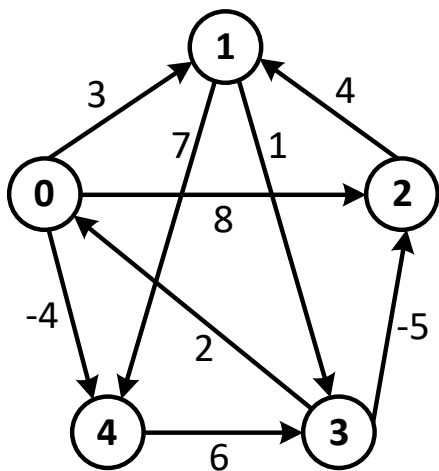
4.3. Floyd-Warshall

Exemplo 2

```

floydWarshall(W)
1  n = quantidade de linhas de W;
2  D(0) = W;
3  for k = 0 até n do
4      for v = 0 até n do
5          for u = 0 até n do
6              dvu(k) = menor(dvu(k-1), dvk(k-1) + dku(k-1));
7          end
8      end
9  end
10 return D(n)

```



$$D^{(0)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \end{matrix}$$

$$D^{(1)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \end{matrix}$$

$$D^{(2)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \end{matrix}$$



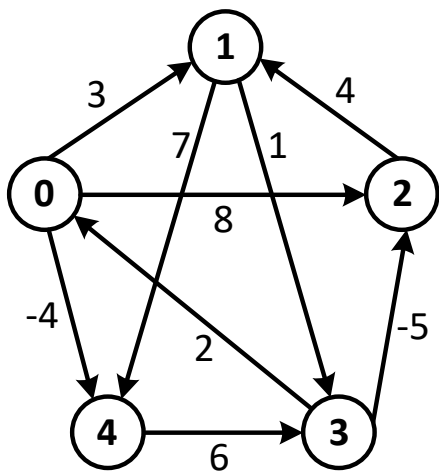
4.3. Floyd-Warshall

Exemplo 2

```

1  floydWarshall(W)
2  n = quantidade de linhas de W;
3  D(0) = W;
4  for k = 0 até n do
5      for v = 0 até n do
6          for u = 0 até n do
7              dvu(k) = menor(dvu(k-1), dvk(k-1) + dku(k-1));
8          end
9      end
10 return D(n)

```



$$D^{(0)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \end{matrix}$$

$$D^{(3)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \end{matrix}$$

$$D^{(1)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \end{matrix}$$

$$D^{(2)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \end{matrix}$$



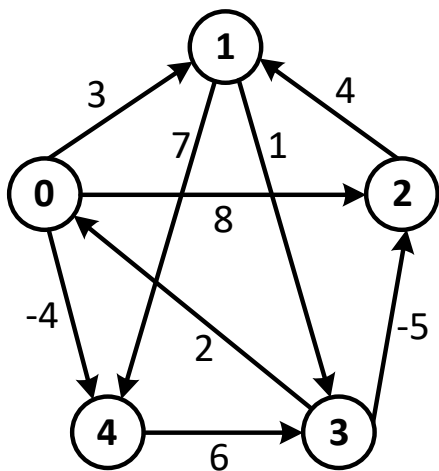
4.3. Floyd-Warshall

Exemplo 2

```

floydWarshall(W)
1  n = quantidade de linhas de W;
2  D(0) = W;
3  for k = 0 até n do
4      for v = 0 até n do
5          for u = 0 até n do
6              dvu(k) = menor(dvu(k-1), dvk(k-1) + dku(k-1));
7          end
8      end
9  end
10 return D(n)

```



$$D^{(0)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \end{matrix}$$

$$D^{(3)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \end{matrix}$$

$$D^{(1)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \end{matrix}$$

$$D^{(4)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & \infty & 1 & 6 & 0 \end{pmatrix} \end{matrix}$$

$$D^{(2)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \end{matrix}$$



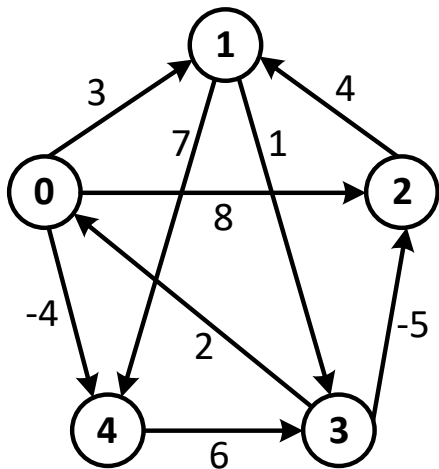
4.3. Floyd-Warshall

Exemplo 2

```

floydWarshall(W)
1  n = quantidade de linhas de W;
2  D(0) = W;
3  for k = 0 até n do
4      for v = 0 até n do
5          for u = 0 até n do
6              dvu(k) = menor(dvu(k-1), dvk(k-1) + dku(k-1));
7          end
8      end
9  end
10 return D(n)

```



$$D^{(0)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \end{matrix}$$

$$D^{(3)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \end{matrix}$$

$$D^{(1)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \end{matrix}$$

$$D^{(4)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & \infty & 1 & 6 & 0 \end{pmatrix} \end{matrix}$$

$$D^{(2)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \end{matrix}$$

$$D^{(5)} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \end{matrix}$$



4.3. Floyd-Warshall

Comentários

- O algoritmo de Floyd-Warshall possui complexidade $O(n^3)$.
- Como o algoritmo não demanda estruturas de dados mais elaboradas, seu código é compacto, sendo **prático para grafos de dimensões moderadas**.



4.3. Floyd-Warshall

Comentários

- O algoritmo de Floyd-Warshall possui complexidade $O(n^3)$.
- Como o algoritmo não demanda estruturas de dados mais elaboradas, seu código é compacto, sendo prático para grafos de dimensões moderadas.

Composição dos Caminhos

Originalmente o algoritmo **não armazenada informações sobre quais vértices compõem os caminhos mais curtos** que foram calculados.

Métodos que podem ser utilizados para construir o caminho:

- Árvores de Caminhos Mais Curtos (*Shortest-Paths Tree*, SPT)
- Construção da matriz predecessora.

Para mais informações: CORMEN, T. “Algoritmos – Teoria e Prática” 3ª Edição, LTC, 2012.

Perguntas? Sugestões?

